Project Report: Supermarket Sales Forecasting API

Prepared By: A. Mohammed Aslam
Organization: The Developers Arena
Project Phase: Month 5 Deliverable

1. Project Overview

This project delivers a production-ready, containerized machine learning pipeline for time-series forecasting. The core objective is to accurately predict next-day supermarket sales based on a rolling 7-day historical window. The solution encompasses data exploration, predictive modeling using a Long Short-Term Memory (LSTM) neural network, and deployment via a RESTful API.

2. Technical Stack

- **Machine Learning & Data Processing:** TensorFlow (Keras), Scikit-learn, NumPy, Pandas
- **API Framework:** FastAPI, Uvicorn, Pydantic
- **Deployment & Containerization:** Docker, Docker Desktop
- **Environment:** Windows Subsystem for Linux (WSL 2)

3. Directory Structure

The repository is organized according to software engineering best practices, optimized for scalability and production deployment:

- `api/` : Contains `main.py`, the core FastAPI application utilizing dynamic relative pathing for artifact retrieval.
- `data/` : Houses the raw transactional datasets (`supermarket_sales.csv`, `customer_churn.csv`).
- `models/` : Stores the serialized deployment artifacts (`lstm_sales_model.h5` and `sales_scaler.pkl`).
- `notebooks/` : Contains Jupyter notebooks documenting the experimental workflow (`01_time_series_exploration.ipynb` and `02_lstm_forecasting.ipynb`).
- `Dockerfile` & `requirements.txt` : Infrastructure-as-code files defining the isolated Linux container environment and necessary Python dependencies.

4. Machine Learning Workflow

1. **Exploratory Data Analysis (EDA):** The transactional dataset was aggregated into a daily time series. Stationarity was verified using the Augmented Dickey-Fuller (ADF) test, yielding a p-value of 0.0000, confirming the data was stationary and ready for modeling without further differencing.
2. **Model Architecture:** An LSTM neural network was selected due to its high efficacy in capturing sequential and temporal dependencies in time-series data.
3. **Preprocessing & Training:** The data was normalized using Scikit-learn's `MinMaxScaler`. The model was trained over 50 epochs, achieving strong convergence (Final validation loss: ~0.024). The scaler and trained model were serialized directly to the `models/` directory using `joblib` and Keras for downstream deployment.

5. API Specification

The prediction endpoint is built with FastAPI, featuring built-in data validation via Pydantic schemas.

- **Endpoint:** `POST /predict`
- **Input Payload:** Requires exactly 7 days of continuous historical sales data.

```
{
  "past_7_days_sales": [1200.5, 1350.2, 1100.0, 1420.8, 1500.3, 1650.0, 1300.9]
}
```

- **Success Response (200 OK):**

```
{
  "next_day_forecast": 1389.56
}
```

6. Docker Containerization

To eliminate environmental discrepancies and ensure seamless portability, the API was containerized using a `python:3.10-slim` base image.

- **Build Command:** `docker build -t supermarket-sales-api .`
- **Run Command:** `docker run -p 8000:8000 supermarket-sales-api`

The container exposes the API on local port 8000, dynamically injecting the required Scikit-learn and TensorFlow dependencies defined in `requirements.txt`, and successfully serving the interactive OpenAPI documentation.